

TryHackMe- Volatility

Volatility is a free memory forensics tool developed and maintained by Volatility Foundation, commonly used by malware and SOC analysts within a blue team or as part of their detection and monitoring solutions. Volatility is written in Python and is made up of python plugins and modules designed as a plug-and-play way of analyzing memory dumps.

Volatility is available for Windows, Linux, and Mac OS and is written purely in Python.

Extracting a memory dump can be performed in numerous ways, varying based on the requirements of your investigation. Listed below are a few of the techniques and tools that can be used to extract a memory from a bare-metal machine.

- **FTK Imager**
- **Redline**
- **Dumplt.exe**
- **win32dd.exe / win64dd.exe**
- **Memoryze**
- **FastDump**

When using an extraction tool on a bare-metal host, it can usually take a considerable amount of time; take this into consideration during your investigation if time is a constraint.

For virtual machines, gathering a memory file can easily be done by collecting the virtual memory file from the host machine's drive. This file can change depending on the hypervisor used; listed below are a few of the hypervisor virtual memory files you may encounter.

- **VMWare - .vmem**
- **Hyper-V - .bin**
- **Parallels - .mem**
- **VirtualBox - .sav file **this is only a partial memory file***

Exercise caution whenever attempting to extract or move memory from both bare-metal and virtual machines.

Listing Processes and Connections

Five different plugins within Volatility allow you to dump processes and network connections, each with varying techniques used. The most basic way of listing processes is using `pslist`; this plugin will get the list of processes from the doubly-linked list that keeps track of processes

in memory, equivalent to the process list in task manager. The output from this plugin will include all current processes and terminated processes with their exit times.

Syntax: `python3 vol.py -f <file> windows.pslist`

The most basic way of listing processes is using `pslist`; this plugin will get the list of processes from the doubly-linked list that keeps track of processes in memory, equivalent to the process list in task manager. The output from this plugin will include all current processes and terminated processes with their exit times.

Syntax: `python3 vol.py -f <file> windows.pslist`

Some malware, typically rootkits, will, in an attempt to hide their processes, unlink itself from the list. By unlinking themselves from the list you will no longer see their processes when using `pslist`. To combat this evasion technique, we can use `psscan`; this technique of listing processes will locate processes by finding data structures that match `_EPROCESS`. While this technique can help with evasion countermeasures, it can also cause false positives.

Syntax: `python3 vol.py -f <file> windows.psscan`

The third process plugin, `pstree`, does not offer any other kind of special techniques to help identify evasion like the last two plugins; however, this plugin will list all processes based on their parent process ID, using the same methods as `pslist`. This can be useful for an analyst to get a full story of the processes and what may have been occurring at the time of extraction.

Syntax: `python3 vol.py -f <file> windows.pstree`

Now that we know how to identify processes, we also need to have a way to identify the network connections present at the time of extraction on the host machine. `netstat` will attempt to identify all memory structures with a network connection.

Syntax: `python3 vol.py -f <file> windows.netstat`

The last plugin we will cover is `dlllist`. This plugin will list all DLLs associated with processes at the time of extraction. This can be especially useful once you have done further analysis and can filter output to a specific DLL that might be an indicator for a specific type of malware you believe to be present on the system.

Syntax: `python3 vol.py -f <file> windows.dlllist`

Volatility Hunting and Detection Capabilities

Volatility offers a plethora of plugins that can be used to aid in your hunting and detection capabilities when hunting for malware or other anomalies within a system's memory.

The first plugin we will be talking about that is one of the most useful when hunting for code injection is `malfind`. This plugin will attempt to identify injected processes and their PIDs along with the offset address and a Hex, Ascii, and Disassembly view of the infected area. The plugin

works by scanning the heap and identifying processes that have the executable bit set `RWE` or `RX` and/or no memory-mapped file on disk (file-less malware).

Based on what `malfind` identifies, the injected area will change. An MZ header is an indicator of a Windows executable file. The injected area could also be directed towards shellcode which requires further analysis.

Syntax: `python3 vol.py -f <file> windows.malfind`

Volatility also offers the capability to compare the memory file against YARA rules. `yarascan` will search for strings, patterns, and compound rules against a rule set. You can either use a YARA file as an argument or list rules within the command line.

Syntax: `python3 vol.py -f <file> windows.yarascan`

Advanced Memory Forensics

The first evasion technique we will be hunting is hooking; there are five methods of hooking employed by adversaries, outlined below:

- SSDT Hooks
- IRP Hooks
- IAT Hooks
- EAT Hooks
- Inline Hooks

We will only be focusing on hunting SSDT hooking as this one of the most common techniques when dealing with malware evasion and the easiest plugin to use with the base volatility plugins.

The `ssdt` plugin will search for hooking and output its results. Hooking can be used by legitimate applications, so it is up to you as the analyst to identify what is evil. As a brief overview of what SSDT hooking is: `SSDT` stands for *System Service Descriptor Table*; the Windows kernel uses this table to look up system functions. An adversary can hook into this table and modify pointers to point to a location the rootkit controls.

There can be hundreds of table entries that `ssdt` will dump; you will then have to analyze the output further or compare against a baseline. A suggestion is to use this plugin after investigating the initial compromise and working off it as part of your lead investigation.

Syntax: `python3 vol.py -f <file> windows.ssdt`

Adversaries will also use malicious driver files as part of their evasion. Volatility offers two plugins to list drivers.

The `modules` plugin will dump a list of loaded kernel modules; this can be useful in identifying active malware. However, if a malicious file is idly waiting or hidden, this plugin may miss it.

This plugin is best used once you have further investigated and found potential indicators to use as input for searching and filtering.

Syntax: `python3 vol.py -f <file> windows.modules`

The `driverscan` plugin will scan for drivers present on the system at the time of extraction. This plugin can help to identify driver files in the kernel that the `modules` plugin might have missed or were hidden.

As with the last plugin, it is again recommended to have a prior investigation before moving on to this plugin. It is also recommended to look through the `modules` plugin before `driverscan`.

Syntax: `python3 vol.py -f <file> windows.driverscan`

In most cases, `driverscan` will come up with no output; however, if you do not find anything with the `modules` plugin, it can be useful to attempt using this plugin.

There are also other plugins listed below that can be helpful when attempting to hunt for advanced malware in memory.

- `modscan`
- `driverirp`
- `callbacks`
- `idt`
- `apihooks`
- `moddump`
- `handles`